

Problem C. Light Bulbs

Input file: `standard input`
Output file: `standard output`

This is an interactive problem with two runs.

Given a village with n houses connected by roads such that there is exactly one simple path between any two houses (i.e., the roads form a tree). The houses are numbered from 0 to $n - 1$. Each house i has a light bulb with a state $a_i \in \{0, 1\}$ and a secret house s is chosen.

No house has more than three roads connected to it.

First Run

In the first run, you know the tree, the array a , and the secret house s . You can change the states of the light bulbs in some houses, setting them to 0 or 1. After all your changes, the states $b_0, \dots, b_{n-1}$ are obtained.

Then, a renumbering of the houses occurs: an unknown permutation p of the set $\{0, \dots, n - 1\}$ is chosen. The old house v receives a new number $p(v)$. In the second run, the states of the light bulbs are given by the array $c_0, \dots, c_{n-1}$ in the new numbering, such that for each v , the equality $c_{p(v)} = b_v$ holds. The roads are also renumbered in the same way: if there was a road between houses u and v in the first run, then in the second run there is a road between houses $p(u)$ and $p(v)$.

You need to implement the function `FirstRun`:

```
vector<bool> FirstRun(int n, vector<pair<int,int> > edges, vector<bool> a, int s)
```

The function should return a `vector<bool>` of size n , containing the new values of the light bulbs $b_0, \dots, b_{n-1}$.

Input for the First Run

- $1 \leq n \leq 2 \cdot 10^5$;
- n is odd number;
- `edges` contains $n - 1$ pairs (u, v) , where $0 \leq u, v \leq n - 1$, defining the roads between the houses;
- it is guaranteed that the given graph is a tree;
- it is guaranteed that the degree of every vertex is at most 3;
- `a` contains n values, where `a[i]` corresponds to a_i ;
- s — the number of the secret house ($0 \leq s \leq n - 1$).

Second Run

In the second run, you are given the renumbered tree and the array c . You need to determine the new number of the secret house, i.e., $p(s)$.

You need to implement the function `SecondRun`:

```
int SecondRun(int n, vector<pair<int,int> > edges, vector<bool> c)
```

The function should return an integer — the new number of the secret house.

Input for the Second Run

- $1 \leq n \leq 2 \cdot 10^5$;
- n is odd number;
- **edges** contains $n - 1$ pairs (u, v) , where $0 \leq u, v \leq n - 1$, in the new numbering;
- the order of **edges** is different with initial order of **edges** before the renumbering;
- it is guaranteed that the given graph is a tree;
- it is guaranteed that the degree of every vertex is at most 3;
- **c** contains n values, where **c**[**i**] corresponds to c_i .

Notes

No information can be saved between the two runs of the program, except for the information encoded by you in the states of the light bulbs. The use of global variables is allowed, but their values are not preserved in the second run; incorrect memory handling may result in a **Wrong Answer** or **Runtime Error** verdict.

Scoring

A solution is considered correct if the function **SecondRun** always returns the correct answer.

Let K be the number of houses where the light bulb value changed in the first run, that is, the number of indices i such that $b_i \neq a_i$. (Equivalently: this is the number of positions where the initial state differs from the returned **FirstRun** array.)

Let mxK be the maximum K across all tests of the subtask.

The problem contains 2 subtasks.

Subtask	Additional Constraints	Points
0	Examples	0
1	Each vertex has degree at most 2, $mxK \leq 20$	21
2	No additional constraints	79

In the second subtask, points are awarded based on the value of mxK :

- $10^5 < mxK \leq 2 \cdot 10^5$ (5 Points)
- $120 < mxK \leq 10^5$ (10 Points)
- $40 < mxK \leq 120$ (20 Points)
- $21 < mxK \leq 40$ (12 Points)
- $18 < mxK \leq 21$ (12 Points)
- $mxK \leq 18$ (20 Points)